

Spark vs. Ray: The Data Engineering Duel

DA3408 Assignment 3 | Aramati Chiru Thejaswi (DA24B036) | 4 October 2026

Summary. The same cleaning pipeline was implemented in PySpark and Ray Data and run on a two-worker cluster of each framework over 20 NYC Yellow Taxi Parquet files (2025-01 to 2026-08, about 1.3 GB compressed). Both produced **57,068,386 rows** and the outputs match exactly, checked value by value. In the final comparable runs, **Spark finished in 223.89 s and Ray in 248.58 s**. Spark was also the only framework that completed the shuffle-heavy variants (global de-duplication and a shuffle join) on the 5.8 GiB test machine.

Winner for this experiment: Apache Spark. Each number comes from a single run; see the limitations.

1. Setup

	Spark	Ray
Version	Apache Spark 4.2.0 (standalone)	Ray 2.58.0, Ray Data
Cluster	1 master + 2 workers (2 cores, 1 GB each), started manually with spark-class; Master UI on port 8080, application UI on 4040	1 head node (0 CPU) + 2 worker nodes (2 CPU each), started manually with ray start; Dashboard on port 8265
Machine	One VirtualBox VM: 4 cores, 5.8 GiB RAM, 4 GiB swap. Both clusters ran on this single VM, one at a time, so workers share one host and there is no real network between them.	
Data	20 Parquet files plus taxi_zone_lookup.csv (265 zones), read from one local folder.	

2. Pipeline (identical in both)

- **Ingest:** read all 20 files, selecting 9 explicit columns. Files from 2026-06 onward carry an extra request_source column, so explicit selection keeps the schemas aligned.
- **Cleanse:** drop nulls; keep rows with dropoff after pickup, duration at most 6 h, 0 < distance at most 100 miles, passengers at least 1; add duration_sec, pickup_date and pickup_hour (UTC). Exact-duplicate removal is available behind a --dedup flag.
- **Transform:** two inner joins with the zone table (pickup and dropoff), then a Python UDF avg_speed_mph = round(trip_distance / (duration_sec / 3600), 2).
- **Export:** Parquet (8.5 GiB written by Ray; Spark produced the same row count).

3. Pipeline parity (25-point criterion)

A DuckDB script compared the two output folders without loading them into memory. All checks passed:

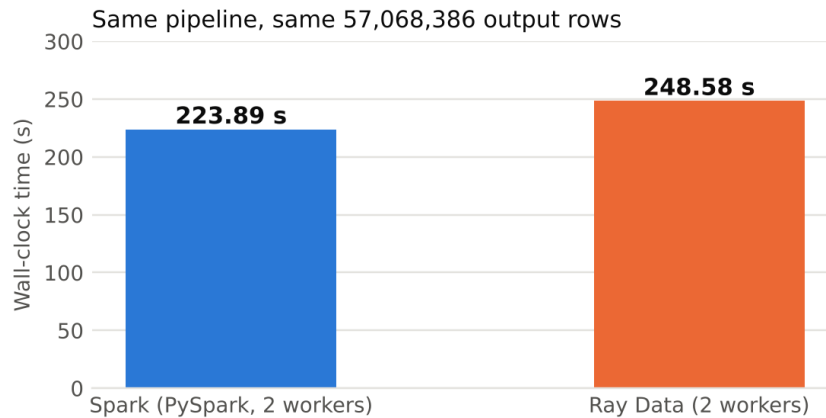
Check	Spark	Ray	Result
Row count	57,068,386	57,068,386	match
Columns (17)	17	17	match
Per-column nulls, min, max (17 columns)	-	-	all match
Order-independent whole-row checksum	526362313234000694607215609	526362313234000694607215609	identical

Sums of floating-point columns differ only by tiny rounding (relative differences below 1e-9), expected from a different summation order, and the row-level match is exact. The parity report is in the repository.

Data-quality observations: one month contained 540,149 of 3,475,226 rows (15.5%) with nulls in five columns at once; trip rows with dropoff before pickup, zero distances and distances up to 276,423 miles were present and removed; pickups dated as early as 2007 survive because the specification has no date filter; no exact duplicates were found (Ray without de-duplication matched Spark with it).

4. Execution time

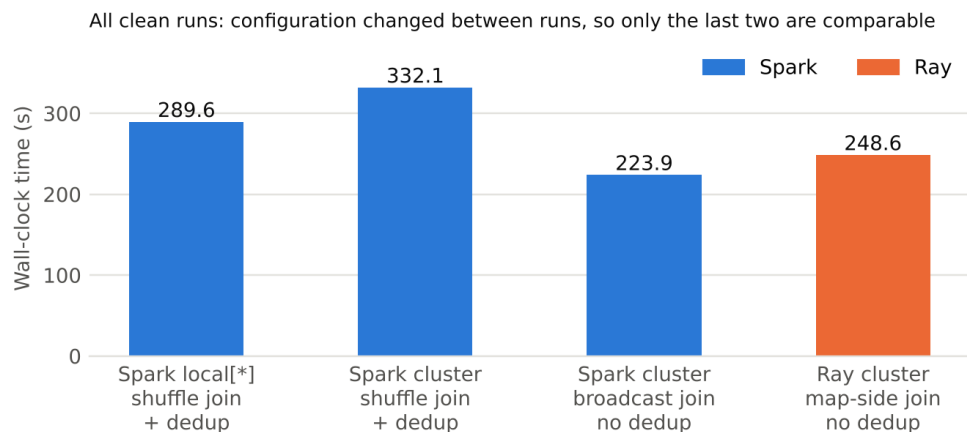
Wall-clock time covers ingestion to export. Both scripts use the same logic, the same inner joins (a small zone table joined without a shuffle) and no de-duplication in the final runs.



Run	Configuration	Wall-clock	Rows
Spark cluster (final)	2 workers, broadcast join, no dedup, UDF on	223.89 s	57,068,386
Ray cluster (final)	2 workers, map-side join, no dedup, UDF on	248.58 s	57,068,386

On these single runs Spark was 24.7 s faster, about 10% less time (Ray took about 11% longer).

Earlier clean Spark runs used a different configuration and are shown only to document how the pipeline evolved, not as a comparison:



A Spark run of 475.92 s is excluded: a second copy of the script was accidentally started during it and competed for CPU and memory. The Spark time dropped from 332.08 s to 223.89 s after removing de-duplication and using a broadcast join; the two changes were made together, so their individual effects were not separated.

5. What happened to the shuffle-heavy variants in Ray

The first Ray version used Ray Data's hash-shuffle join and a global group-by to de-duplicate on nine columns. On the 5.8 GiB VM the dashboard showed the memory budget full (3.9 of 3.9 GiB), the read stage back-pressured with no tasks running, and the object store at 470 of 477 MiB. The group-by was at 10% after about 12 minutes with an estimated 1 h 25 min remaining and was stopped; a later hash-join attempt triggered repeated out-of-memory worker kills. Spark completed the equivalent de-duplication plus shuffle join in 332.08 s on the same machine (table above). This is a qualitative observation from the runs, not a controlled benchmark.

6. Resource utilization

What was observed, and what was not captured:

- **Spark (Executors page, final run):** 2 executors with 2 cores each, 37 tasks, **0 failed tasks**, garbage-collection time 21 s out of 24 min of task time (about 1.5%), so the 1 GB executors were not under memory pressure.
- **Ray (dashboard, earlier attempt):** CPU about 82% on each of the three nodes while a job ran, with object stores of 286 MB (head) and 477 MB (each worker). The dashboard's memory and CPU columns report the whole VM, so the three nodes show nearly identical values.
- **Not captured:** clean peak CPU and peak memory logs (top) for the two final runs. The only top log was recorded during the contaminated Spark run and is not used. Peak utilization is therefore not tabulated here; this is a gap in the evidence, listed in the limitations.

7. Python UDF overhead

Not measured in isolation. The UDF is fused with reading and filtering into one stage in both frameworks (Spark's largest stage, 3.5 min over 16 tasks, contains it), so its cost cannot be read from the UI. A with-and-without-UDF comparison was prepared (`--no-udf` flag in both scripts) but not run. What follows is architectural reasoning, not a measured result:

- **Spark:** a Python UDF runs in separate Python worker processes. Every row is serialized from the JVM to Python and back, and the optimizer treats the function as a black box, so no filter or projection can be pushed through it.
- **Ray:** `map_batches` runs the function directly in a Ray worker process on a pandas or Arrow batch. There is no JVM boundary, and the data is already in the Python ecosystem that model training uses.
- **Consequence for this result:** the UDF here is cheap arithmetic, and Spark was still faster overall. The Python-native advantage of Ray would matter more for heavier per-row Python (feature code, model inference), which this pipeline does not exercise.

8. Performance Tuning Note (AI assistance)

An LLM (Claude) helped draft the pipeline logic and suggested the following changes. The cluster configuration and network setup were done manually, and every timing in this report is copied from my own runs and the UI screenshots.

- **Ray join:** replace Ray Data's hash-shuffle join with a map-side join. The 265-row zone table is loaded once and joined to each batch with Arrow, so no shuffle is needed. Reason: the shuffle join's aggregators reserved about 800 MB each and caused out-of-memory kills on the 5.8 GiB VM. The result is still an inner join with the same output.
- **Spark join:** add an explicit broadcast hint to the same small table and remove the setting that forced shuffle joins, so both frameworks do equivalent work.
- **De-duplication:** made optional (`--dedup`), because the global group-by was infeasible in Ray on this machine and removed no rows (the row count with it, in Spark, equals the count without it, in Ray).
- **Parity fix:** read the zone CSV so that the literal text N/A stays a string in both frameworks (a default CSV reader turns it into null in Ray but not in Spark), and cast timestamps and zone IDs to the same types.

9. Limitations

- Each reported time is **one run** per framework. No repeats, so no spread, and a 25 s difference may be inside normal run-to-run noise.
- Both frameworks ran on **one VM**, one at a time. Two workers share the same CPU, memory and disk, so shuffle and network costs of a real cluster are not represented.
- The operating-system file cache may have been warmer for later runs than earlier ones; the order of runs was not randomized.
- Peak CPU/memory and isolated UDF overhead were not measured (sections 6 and 7).

10. Conclusion: which framework, for which project

Winner in this experiment: Apache Spark. It finished the final pipeline about 10% faster (223.89 s vs 248.58 s) and was the only one to complete the shuffle-heavy variants on the 5.8 GiB machine. Because the evidence is a single run per framework on a single VM, the speed gap is suggestive, while the robustness difference was large and repeated across several attempts.

	BI-first project	AI-first project
Recommendation	Spark. Structured, SQL-style cleaning, joins and aggregations, large shuffles, mature optimizer, and it stayed stable under memory limits.	Ray. Python-native pipelines whose output feeds training or inference in the same cluster, with batch functions that run without a JVM boundary and can use GPUs.
Evidence from this work	Spark completed global de-duplication and a shuffle join (332.08 s) where Ray's equivalents stalled or were killed; final run was faster.	Ray's map_batches ran the Python function in-process; but the UDF cost was not isolated, so this is architectural reasoning, not a measured advantage.

11. Evidence

Spark Master UI during the final run: 2 workers ALIVE, all 4 cores in use, application spark_clean RUNNING.

Spark Master at spark://127.0.0.1:7077

URL: spark://127.0.0.1:7077
REST URL: spark://127.0.0.1:6066 (cluster mode)
Workers: 2 Alive, 0 Dead, 0 Decommissioned, 0 Unknown
Cores in use: 4 Total, 4 Used
Memory in use: 2.0 GiB Total, 2.0 GiB Used
Resources in use:
Applications: 1 Running, 0 Completed
Drivers: 0 Running (0 Waiting), 0 Completed (0 Killed, 0 Failed, 0 Error, 0 Relaunching)
Status: ALIVE (Environment, Log)

Workers (2)

Worker Id	Address	State	Cores	Memory
worker-20261004141701-127.0.0.1-43469	127.0.0.1:43469	ALIVE	2 (2 Used)	1024.0 MiB (1024.0 MiB Used)
worker-20261004141738-127.0.0.1-42619	127.0.0.1:42619	ALIVE	2 (2 Used)	1024.0 MiB (1024.0 MiB Used)

Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20261004142035-0000	(kill) spark_clean	4	1024.0 MiB		2026/10/04 14:20:35	aramati-chiru-thejaswi	RUNNING	41 s

Completed Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------

Ray Dashboard, Cluster tab: 3 nodes ALIVE (1 head, 2 workers).

Node Statistics

TOTAL x3 ALIVE x3

Node List

Host / Worker Process name	State	State Message	ID	IP / PID	Actions	CPU	Memory	Object Store Memory	Disk(root)	Sent	Receive
aramati-chiru-thejaswi-VirtualBox	ALIVE	-	5a8a8...	10.0.2.15 (Head)	Log	19.9%	3.91GB/5.78GB (67.6%)	0.0000B/286.10MB (0.0%)	32.08GB/48.91GB (69.1%)	1.62KB/s	540.792t
aramati-chiru-thejaswi-VirtualBox	ALIVE	-	29b91...	10.0.2.15	Log	18.9%	3.91GB/5.78GB (67.7%)	0.0000B/476.84MB (0.0%)	32.08GB/48.91GB (69.1%)	1.68KB/s	533.514t
aramati-chiru-thejaswi-VirtualBox	ALIVE	-	b8590...	10.0.2.15	Log	17.5%	3.90GB/5.78GB (67.5%)	0.0000B/476.84MB (0.0%)	32.08GB/48.91GB (69.1%)	1.15KB/s	456.098t